PROJECT REPORT

Malaria Detection using Deep Learning

Project Design Phase-II

Team Members:

Konarksinh Solanki (Team Lead) | Prachin Nayak | Rudra Patel

1. Project Title

Malaria Detection using Deep Learning

A CNN-based deep learning web application for automated classification of blood smear cell images as Parasitized (malaria-infected) or Uninfected.

---

2. Project Description

System Overview

The Malaria Detection system is a full-stack deep learning application that automates the diagnosis of malaria from microscopic blood smear images. It leverages Convolutional Neural Networks (CNNs) — specifically Custom CNN, MobileNetV2, and EfficientNetB0 — trained on the NIH malaria cell image dataset (27,558 images). The system is deployed as a Flask web application where users upload cell images and receive instant binary predictions with confidence scores and Grad-CAM heatmap visualizations.

Problem Domain

Malaria remains one of the world's deadliest diseases, causing approximately 600,000 deaths annually (WHO, 2023). Traditional diagnosis via manual microscopic examination of blood smears is slow (1–2 hours per sample), labor-intensive, and requires expert pathologists who are often unavailable in rural and remote areas of sub-Saharan Africa, South Asia, and Southeast Asia.

Core Technology Used

- Deep Learning: Custom CNN, MobileNetV2, EfficientNetB0 (TensorFlow/Keras)
- Computer Vision: OpenCV for image preprocessing
- Web Framework: Python Flask with Jinja2 templating

Key Capabilities

- Binary classification: Parasitized vs. Uninfected cell detection with >95% accuracy
- Confidence score output (0–100%) for every prediction
- Grad-CAM heatmap visualization highlighting infected regions in cell images
- Batch image upload for mass screening scenarios
- PDF report generation for patient records
- Admin dashboard with prediction history and model performance metrics

Target Users

- Healthcare workers and lab technicians in rural/remote clinics
- Hospital physicians in malaria-endemic regions
- Medical researchers and data scientists studying malaria detection
- NGOs and public health agencies conducting mass screening programs

Real-world Relevance

With >240 million malaria cases annually and a critical shortage of pathologists in endemic regions, an automated, fast, and accessible AI diagnostic tool can directly save lives. The system reduces diagnosis time from hours to seconds, enabling timely treatment decisions in resource-limited settings.

---

3. Application Scenarios / Use Cases

3.1 Rural Clinic Screening

A rural health worker at a community clinic uploads blood smear cell images captured by a basic microscope. The system instantly classifies each cell image as Parasitized or Uninfected and provides a confidence score. The healthcare worker uses the AI-assisted result to make a treatment decision without waiting for a pathologist's review.

3.2 Hospital Mass Screening

During malaria outbreak season, a district hospital needs to screen hundreds of blood samples rapidly. Technicians use the batch upload feature to submit multiple images simultaneously. The system processes all images sequentially and generates a downloadable PDF report with results for each patient, enabling the hospital to prioritize high-risk cases.

3.3 Research & Model Benchmarking

A medical researcher uses the evaluation dashboard to compare the performance of Custom CNN, MobileNetV2, and EfficientNetB0 models on the NIH malaria dataset. The researcher analyzes accuracy, precision, recall, F1-score, and ROC-AUC metrics, and uses confusion matrices to understand which model minimizes false negatives (missed infections).

3.4 NGO Field Deployment

An NGO conducting malaria screening in remote villages deploys the Flask application on a local server with no internet connectivity (offline mode). Community health workers upload images using tablets connected to the local server, enabling AI-assisted malaria screening even in areas without internet access.

---

4. Technical Architecture Overview

4.1 High-level Architecture

The system follows a 4-layer architecture: Presentation → Application/Model → Data → Infrastructure.

- Presentation Layer: Flask + Jinja2 web interface for image upload, results display, and admin panel
- Application / Model Layer: Flask REST API, image preprocessing module, CNN inference engine
- Data Layer: NIH cell image dataset, trained model weights (.keras)

4.2 Component Interaction

User (Browser) → Flask Web App (upload) → Image Preprocessor (resize/normalize) → CNN Model (inference) → Result Generator (confidence) → Results Page (display)

4.3 Frontend Layer

HTML5/CSS3 with Bootstrap 5 for responsive design. Jinja2 templates render dynamic content. JavaScript handles file preview and form submission. No separate frontend framework — all server-side rendered for simplicity.

4.4 Backend Layer

Python Flask REST API with Routes handle image upload (/predict, result retrieval (/result/<id>), and health check (/health).

---

5. Prerequisites

5.1 Software Requirements

- Python 3.9 or higher
- pip (Python package manager)
- Git (version control)
- IDE: VS Code / PyCharm
- Browser: Chrome / Firefox (for accessing the web interface)

5.2 Libraries / Frameworks / Dependencies

Package	Version	Purpose
tensorflow	2.13.x	Deep learning framework for CNN model training and inference
keras	included in TF	High-level neural network API for model building
flask	2.3.x	Lightweight web framework for REST API and web interface
opencv-python	4.8.x	Image reading, resizing, normalization
numpy	1.24.x	Numerical computing for image array manipulation
scikit-learn	1.3.x	Evaluation metrics: accuracy, precision, recall, F1, ROC-AUC
matplotlib	3.7.x	Plotting training curves, confusion matrix, ROC curves
pillow	10.x	Image format handling and conversion

5.3 Hardware Requirements

- Minimum: 4GB RAM, 2-core CPU, 10GB disk (for development and inference)
- Recommended: 8GB RAM, 4-core CPU, 20GB disk (for training and production)
- GPU (Optional): NVIDIA GPU with CUDA 11.x for faster training (NVIDIA GTX 1060 or better)

---

6. Prior Knowledge Required

- Python programming: Functions, OOP, file I/O, virtual environments
- Deep Learning fundamentals: Neural networks, CNNs, backpropagation, activation functions
- Computer Vision: Image preprocessing, data augmentation, feature maps
- Transfer Learning: Fine-tuning pre-trained models (MobileNetV2, EfficientNetB0)
- Flask basics: Routes, request handling, Jinja2 templates, REST API design
- Git/GitHub: Version control, branching, pull requests, Actions CI/CD

---

7. Project Objectives

Technical Objectives

- Build and train a Custom CNN model achieving >90% accuracy on NIH malaria dataset
- Fine-tune MobileNetV2 and EfficientNetB0 using transfer learning
- Develop a Flask REST API with endpoints for single-image and batch-image prediction
- Build a web interface for image upload, result display, and PDF report generation

Performance Objectives

- Achieve classification accuracy >95% and ROC-AUC >0.98 on validation set
- Achieve precision >94% and recall >96% (minimize false negatives — missed infections)
- Model inference time <5 seconds per image on standard CPU hardware
- Web application response time <3 seconds for single prediction requests

Learning Outcomes

- Gain hands-on experience in CNN architecture design and transfer learning
- Understand end-to-end ML pipeline from data preprocessing to model deployment
- Learn to build and deploy production-grade Flask applications

8. System Workflow

1. User opens the Flask web application in a browser and navigates to the upload page.
2. User selects or drags-and-drops a blood smear cell image (PNG/JPG, max 10MB) onto the upload form.
3. Flask backend validates the file type and size. Invalid files return an error response.
4. Valid image is read by OpenCV, resized to 128×128 pixels, and pixel values normalized to [0, 1].
5. Preprocessed image tensor is passed to the loaded CNN model for inference.
6. Model outputs a probability score: values >0.5 → Parasitized, ≤0.5 → Uninfected.
7. Flask returns a JSON response with: result label, confidence score (%).
8. Results page renders the prediction, confidence score, original image

9. Milestone 1: Requirement Analysis & System Design

9.1 Problem Definition

Malaria diagnosis via manual blood smear microscopy is slow, error-prone, and inaccessible in resource-limited settings. The project addresses this by building an automated deep learning system that classifies blood smear cell images with expert-level accuracy and provides explainable results that clinicians can trust.

9.2 Functional Requirements

- FR-1: System shall accept PNG/JPG blood smear images (max 10MB) via web upload
- FR-2: System shall classify each image as Parasitized or Uninfected with a confidence score

9.3 Non-Functional Requirements

- NFR-1: Classification accuracy >95% on NIH malaria validation dataset
- NFR-2: Inference time <5 seconds per image on CPU
- NFR-3: Web app response time <3 seconds

9.4 System Design Decisions

- CustomCNN selected as default inference model — lightweight, high accuracy, CPU-friendly
- Flask chosen over Django for simplicity and lower overhead for an ML-serving application

9.5 Technology Stack Selection Justification

- TensorFlow/Keras: Industry-standard DL framework with excellent pretrained model support
- CustomCNN: Optimized for mobile/low-resource inference with state-of-the-art accuracy
- Flask: Minimal framework ideal for ML APIs — no overhead of full-stack frameworks
- OpenCV: Best-in-class library for image preprocessing and Grad-CAM overlay